21CSE525T – AGENT TECHNOLOGY

ASSIGNMENT 5.1

Agent Authentication and Host Integrity Verification

CipherMesh: The Privacy Control Plane for Enterprise AI

Submitted by

KUMAR WAIBHAV AKSHAT [RA2212704010027]

M.Tech (Integrated) Computer Science & Engineering
with specialization in Data Science

Under the Guidance of

Dr. Prabhu Kavin B

Assistant Professor
Department of Data Science and Business Systems

DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203

NOVEMBER 2026

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Objective 1
1.2 The problem in CipherMesh terms 1
1.3 How this unit builds on Units 1 to 3 2
1.4 Threat model and scope 2
2 ASSIGNMENT REQUIREMENTS AND TRACEABILITY 2
3 DESIGN 3
3.1 Trust architecture 3
3.2 Trust store and key model 3
3.3 SignedCommand envelope and canonical signing input 4
3.4 The thirteen-checkpoint verification pipeline 5
3.5 Host integrity verification 6
3.6 Replay, freshness, lockout and connection limits 7
3.7 Fail-closed replies and audit hygiene 8
4 IMPLEMENTATION 8
4.1 Module map 8
4.2 Selected code 9
4.3 Channels and the trusted host environment 10
4.4 Execution on the real engine 10
5 VERIFICATION RUN 11
5.1 Set-up 11
5.2 Legitimate traffic is accepted 11
5.3 Breach simulation: every unauthorised request is blocked 11
5.4 A blocked unauthorised request in the runtime log 14
5.5 State is unchanged and prompts are still processed correctly 15
5.6 Host integrity results 15
5.7 Timeline and cost 16
6 TESTING 17
6.1 Test suite 17
6.2 Edge cases worth noting 18
7 SUBMISSION ANALYSIS 18
7.1 Requirement, evidence and status 18
7.2 Strengths 18
7.3 Honest limitations 19
7.4 How this feeds the Capstone 5.2 19
8 CONCLUSION 19
REFERENCES 20
App. A HOW TO RUN AND FILE LISTING 21
A.1 Commands 21
A.2 Deliverable files 21

LIST OF FIGURES

Figure 3.1 Trust architecture: hosts, channels, verifier and engine....3
Figure 3.2 Ordered verification pipeline with each reject code....5
Figure 3.3 Host integrity: manifest, self-attestation and peer evidence....7
Figure 5.1 Attack requests stopped by each checkpoint (labels give attack ids)....13
Figure 5.2 Signed command accepted versus forged command rejected at C8....14
Figure 5.3 Breach simulation timeline: verifier latency per request....16
Figure 5.4 Per-checkpoint cost (left) and early rejection versus acceptance (right)....17

LIST OF TABLES

Table 1.1 Shared-key HMAC (Unit 2.2) versus per-host Ed25519 (this unit)....1
Table 2.1 Traceability from the assignment brief (quoted verbatim) to code and evidence....2
Table 3.1 Fields of a host entry in trusted_hosts.json....4
Table 3.2 SignedCommand header fields....4
Table 3.3 Time-based and connection limits....8
Table 4.1 Modules of the ciphermesh/trust subpackage....8
Table 5.1 Legitimate requests (Parts A, C and D)....11
Table 5.2 Attack requests, blocking checkpoint and reason....11
Table 5.3 Prompt decisions after the breach simulation....15
Table 5.4 Host integrity checks (logs/integrity_check.log)....15
Table 5.5 Micro-benchmark results (median of 3,000 runs unless stated)....16
Table 6.1 New tests in tests/trust....17
Table 7.1 Requirement status....18
Table A.1 Deliverable files....21

1 INTRODUCTION

1.1 Objective

This assignment adds cryptographic validation checkpoints to the CipherMesh agent so that no command can change its behaviour unless the command is authentic, unmodified, fresh, sent from a trusted place by a trusted host running trusted code, and permitted for that host. The deliverable is a new, tested `ciphermesh/trust/` subpackage, a secure agent node that uses it on both a TCP network loop and a local system-command channel, a breach simulation that attacks the node with 41 unauthorised requests, and the runtime logs that record every blocked request.

An agent in the sense of Russell and Norvig perceives its environment through sensors and acts through actuators [1]. For CipherMesh, inbound commands are part of what the agent perceives, and some of them (emergency lockdown, new redaction patterns, policy synchronisation) directly change how it acts on every later prompt. A forged percept is therefore as dangerous as a broken detector. This unit protects the percept channel itself.

1.2 The problem in CipherMesh terms

Assignment 2.2 connected CipherMesh nodes with length-prefixed TCP frames and signed each envelope with **HMAC-SHA256** over a shared key, with a freshness window and a replay cache [2]. That design proves that the sender knows the shared key and that the message was not changed in transit. It does not say *which* node sent it, and it has a structural weakness: in HMAC the verification key and the signing key are the same key. Every gateway that can check a command can also create one. Table 1.1 compares the two trust models.

Table 1.1: Shared-key HMAC (Unit 2.2) versus per-host Ed25519 (this unit)

Question	Unit 2.2: shared HMAC key	Unit 5.1: Ed25519 per host
What does a gateway hold?	The one secret key used by every node	Only public keys of other hosts
Can a compromised gateway forge an admin lockdown?	Yes - it can compute a valid MAC for any message	No - it can verify signatures but never produce one
Who sent this command?	Any key holder (all nodes look the same)	A named host; key_id is bound to exactly one host
Revoke one host	Re-key every node in the mesh	Mark one key or host revoked in the trust store
Least privilege	Not possible: one key, one level of trust	Roles per host: admin, observer, peer gateway
Is the sender running trusted code?	Not checked	Attestation digest checked (C12)

Ed25519 is the Edwards-curve signature scheme of RFC 8032 [3], designed by Bernstein et al. for fast, deterministic, side-channel resistant signing [4]. With it, the agent stores only public keys, so stealing the agent's configuration gives an attacker nothing it can sign with. Per-host keys also give each host its own cryptoperiod and revocation, as recommended by NIST SP 800-57 [5]. The change follows the zero-trust principle of NIST SP 800-207: no request is trusted because of where it comes from alone; every request is authenticated and authorised on its own [6].

1.3 How this unit builds on Units 1 to 3

- **Unit 1.1 and 1.2** defined the PEAS model and the reflex policy (R1 to R5). The secure agent still runs exactly that policy; an accepted lockdown sets `AgentState.force_block_all`, which makes `policy.inspect_text` return BLOCK for every prompt.
- **Unit 2.1** gave the threaded runtime; the secure TCP server reuses its bounded worker pattern. **Unit 2.2** gave the frame format (`transport.py`) and the command vocabulary; both are reused unchanged, and the HMAC module stays in place for the legacy path.
- **Units 3.1 and 3.2** (Contract-Net procurement and the constrained utility router) depend on a correct `AgentState`. By keeping that state out of reach of unauthorised commands, this unit protects every routing and residency decision taken later.

1.4 Threat model and scope

The attacker can reach the agent over the network, can send any bytes, can capture and replay earlier traffic, may hold its own Ed25519 key, may have stolen a valid admin key, may control a machine inside the admin subnet, may have a key that has since been rotated out, and may control a host whose code has been modified. Locally, another account may try to plant command files. Out of scope are a full compromise of the agent host itself (an attacker with root on the agent can change anything) and hardware-rooted attestation, which is discussed as a limitation in Section 7.

2 ASSIGNMENT REQUIREMENTS AND TRACEABILITY

Table 2.1 maps every Direction and Deliverable line of the assignment brief to the place where it is met and to the evidence file a reader can open.

Table 2.1: Traceability from the assignment brief (quoted verbatim) to code and evidence

Brief line (verbatim)	Where it is satisfied	Evidence
Objective: Build cryptographic validation checkpoints protecting your agent's data payload from unauthorized access or modification.	`trust/verifier.py` CommandVerifier: L0 + C1-C13, fail closed	`logs/security_runtime.log`; Fig. 3.2
Implement secure payload parsing inside your agent via a signature system (e.g., verifying a sender's public keys before processing commands).	`trust/envelope.py`, `trust/canonical.py`, `trust/keys.py`; C2, C5, C6, C8, C9 run before any action	`tests/trust/test_envelope.py`, `test_verifier.py`, `test_keys.py`
Set up authentication checks verifying that payloads received via network loops or system commands originate from a trusted host environment.	C7 (peer IP / CIDR; spool file owner and mode), C12 attestation, self-attestation in `trust/integrity.py`	`logs/integrity_check.log`; A12, A21, A23, I01, I02
Simulate an untrusted input breach and demonstrate how your agent safely handles or rejects unauthorized commands.	`trust/attack_sim.py`: 41 attack requests over real TCP loopback and the spool channel	`logs/breach_simulation_summary.json` / `.txt`
Deliverables: Cryptographically enhanced agent code module along with verified runtime test logs capturing a blocked unauthorized request event.	`ciphermesh/trust/` (14 modules); "BLOCKED UNAUTHORIZED REQUEST" entries with checkpoint, reason and STATE UNCHANGED	`logs/security_runtime.log`, `security_events.jsonl`, `pytest_results.txt`

3 DESIGN

3.1 Trust architecture

Figure 3.1 shows the parties. Three kinds of trusted host talk to the agent: the security console (role `security-admin`), an observer (role `observer`) and a peer gateway (role `peer-gateway`). Commands arrive by two channels: the TCP network loop and a local system-command spool directory. Inside the agent, the CommandVerifier sits in front of everything. It reads the trust store (public keys, allowed sources, roles, revocations) and the signed release manifest. Only an accepted command reaches the executor, which maps it to an action on the Unit 1 to 3 engine and signs the reply. Every decision is written to a JSONL security audit.

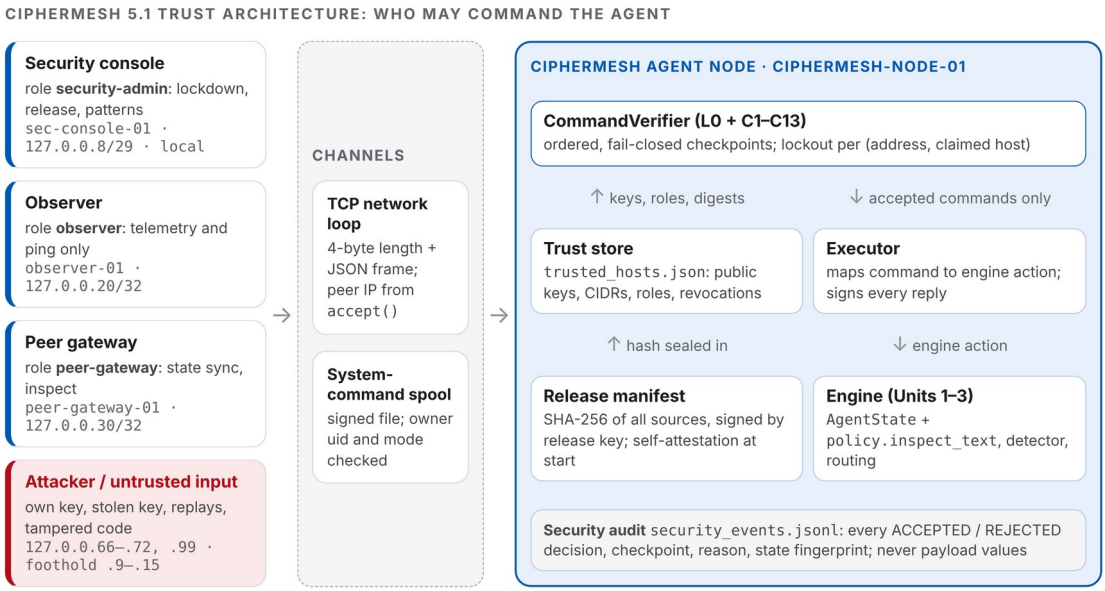


Figure 3.1: Trust architecture: hosts, channels, verifier and engine

3.2 Trust store and key model

Each host owns an Ed25519 key pair. The key id is the first 16 hexadecimal characters of SHA-256 over the 32-byte raw public key, so a key id is short, stable and cannot be chosen freely. Private keys are written as PKCS#8 PEM, encrypted with a passphrase, with file mode 0600; loading refuses a key file that group or others can read, in the same way OpenSSH does. The agent stores only `trusted_hosts.json`, whose fields are listed in Table 3.1. The file is checked against a strict schema (Pydantic models with unknown fields forbidden) plus semantic rules: the key id must match the key, a key may belong to only one host, every role must exist, every CIDR must parse, and the file must not be group- or world-writable.

Table 3.1: Fields of a host entry in trusted_hosts.json

Field	Meaning	Checked at
host_id	Lower-case name, e.g. `sec-console-01`; must equal header.sender_host	C5
roles	`security-admin`, `observer`, `peer-gateway` or `agent`; roles map to command lists	C13
allowed_sources	CIDR blocks such as `127.0.0.8/29`, or `local` for the spool channel	C7
keys[]	key_id, base64 public key, valid_from, valid_until, revoked flag, note	C6, C8
trusted_release_digests	SHA-256 attestation digests of code releases this host may run	C12
valid_from / valid_until / revoked	Validity window of the host itself; a decommissioned host is revoked	C5
revocation_list (top level)	key_id, revoked_at, reason: a small CRL kept next to the keys	C6

Rotation is a two-step process supported by the API: `add_key()` publishes a new key next to the old one, both are valid during an overlap, and `revoke_key()` then revokes the old key and records it in the revocation list. Only that one key stops working; no other host is touched. The provisioned store models this: the security console has a rotated-out key (revoked) and a current key.

3.3 SignedCommand envelope and canonical signing input

A command travels as one frame containing `header`, `payload` and `signature`. Table 3.2 lists the header fields. The signature is Ed25519 over a domain-separation prefix followed by the canonical JSON of the header and payload together.

Table 3.2: SignedCommand header fields

Field	Format	Purpose
version	`cm-sig/1`	Rejects downgrade to other formats (C3)
msg_id / nonce	UUID4 / 128-bit hex	Unique identity of the message; replay cache keys (C11)
sender_host / key_id	host id / 16 hex	Who claims to send it, with which key (C5, C6)
recipient	agent id	Stops a command meant for another node being redirected (C4)
command	one of 7 inbound commands	What is asked for; drives the payload schema and C13
issued_at / expires_at	RFC 3339 UTC, ms	Freshness window and lifetime (C10) [7]
payload_sha256	64 hex	Binds the body to the header (C9)
integrity	64 hex	Sender's code attestation digest, its evidence for C12

Canonical JSON follows the JSON Canonicalization Scheme of RFC 8785 [8]: members sorted by UTF-16 code units, no whitespace, UTF-8 strings with only the mandatory escapes, and numbers in the ECMAScript shortest form. Both sides sign and verify the canonical form of the *parsed* object, so spacing and member order on the wire do not matter, but any change to a value

does. Different domain prefixes are used for commands, agent replies and release manifests, so a signature made for one purpose can never be replayed as another.

```
COMMAND_CONTEXT = b"CipherMesh/SignedCommand/v1\x00"
REPLY_CONTEXT = b"CipherMesh/SignedReply/v1\x00"

def signing_bytes(header: dict, payload: dict, context: bytes = COMMAND_CONTEXT) -> bytes:
    """Exact bytes covered by the Ed25519 signature."""
    return context + canonicalize({"header": header, "payload": payload})
```

Listing: the signing input (ciphermesh/trust/envelope.py)

3.4 The thirteen-checkpoint verification pipeline

Figure 3.2 shows the pipeline. Every raw payload, from either channel, passes through it *before* anything is parsed into an action. The checkpoints run in a fixed order and the first failure ends processing with a stable reason code. L0 implements the lockout of repeated failures.

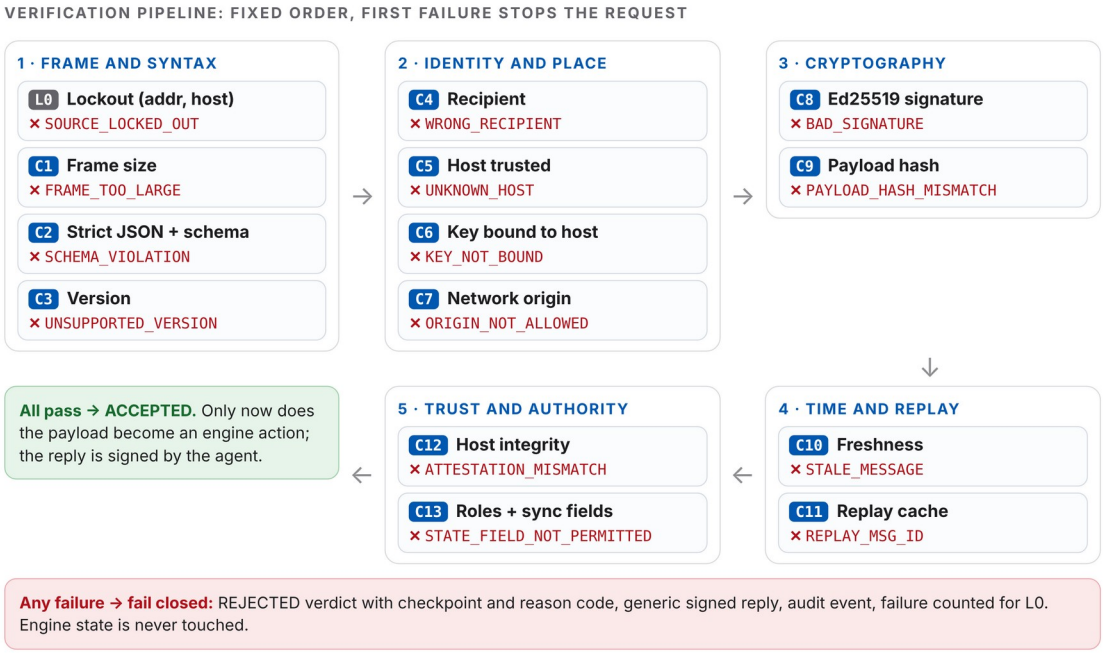


Figure 3.2: Ordered verification pipeline with each reject code

Algorithm 1: CommandVerifier.verify(raw, origin)

1. L0: failures are counted per (peer address, claimed host); a frame with no parsable host counts against (address, -). A request whose key is locked is refused with SOURCE_LOCKED_OUT unless it passes C8, in which case the lock is cleared (see Section 3.6).
2. C1: if the declared frame length is 0 or above 64 KiB (or the local file is not a regular file), reject. The TCP server applies this to the length prefix before reading the body.
3. C2: decode strict UTF-8, parse JSON with duplicate member names and NaN/Infinity refused, validate the envelope and the per-command payload schema with unknown fields forbidden; REPLY is not accepted inbound.
4. C3 and C4: the version must be supported and the recipient must equal this agent id (constant-time compare).
5. C5 and C6: the sender host must exist, not be revoked and be inside its validity window; the key id must be bound to that host, not be revoked or in the revocation list, and be valid now.

6. C7: for TCP the kernel-reported peer IP must lie in one of the host's CIDR blocks; for the spool the host must allow `local`, the file owner must be the agent's service account and the file must not be group- or world-writable.
7. C8: verify the Ed25519 signature with the bound public key over the canonical signing input.
8. C9: recompute SHA-256 of the canonical payload and compare with header.payload_sha256 (constant time).
9. C10: issued_at must be within plus or minus 60 s of local time, the lifetime must be between 0 and 120 s, and the message must not be expired.
10. C11: msg_id and nonce must not be in the replay cache; add both.
11. C12: header.integrity must equal one of the host's trusted release digests (constant time); on failure raise a HOST_INTEGRITY_FAILURE alert.
12. C13: the command must be granted to one of the host's roles. For STATE_SYNC a host without `security-admin` may only send the non-weakening fields `policy_revision`, `policy_version`, `provider_allowlist` and `max_prompt_bytes`; any other field (custom patterns, secret blocking, PII redaction, internal-URL review, lockdown) is refused with STATE_FIELD_NOT_PERMITTED. In SAFE MODE only telemetry and ping are allowed.
13. On any failure: record it against the (address, claimed host) key (five failures in 60 s give a 300 s lockout and a SOURCE_LOCKOUT alert) and return a REJECTED verdict. Any unexpected exception also becomes a rejection (INTERNAL_ERROR): the verifier fails closed.

The order is deliberate. Cheap checks that need no keys (C1 to C4) come first, so floods of garbage cost microseconds. Identity look-ups (C5 to C7) come before the comparatively expensive signature check, so traffic from unknown hosts or wrong networks never reaches Ed25519. The replay cache (C11) is only updated after the signature has been proven, which means an attacker cannot fill the cache with forged ids to block real messages. Authorisation (C13) is decided last, on facts that are now authenticated. C13 checks fields as well as commands because the Unit 2.2 engine accepts any policy field in a newer STATE_SYNC revision; without the field rule a peer gateway could switch off secret blocking (attack A26).

C8 and C9 are both needed. Because the signature covers the payload itself, an attacker who edits the payload is caught at C8 even if they also recompute the digest. C9 catches a different fault: a sender holding a valid key that binds the wrong digest, for example a buggy or half-compromised client. Downstream code can then trust `payload_sha256` as a short identifier of the body.

3.5 Host integrity verification

Figure 3.3 shows how host integrity is checked. The roles follow the IETF RATS architecture [9]: the release manifest holds the *reference values*, each host's attestation digest is its *evidence*, and the agent is the *verifier*. The approach addresses OWASP's "software and data integrity failures" category, where code or configuration is changed without detection [10].

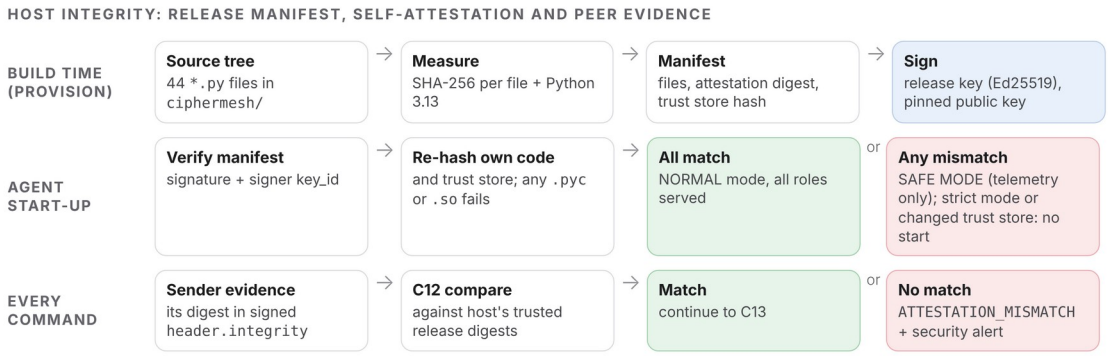


Figure 3.3: Host integrity: manifest, self-attestation and peer evidence

Algorithm 2: build, self-attest and check peers

1. Build: compute SHA-256 of every `*.py` file in the `ciphermesh` package and record the Python minor version. The attestation digest is SHA-256 over the canonical JSON of {release, python, files}.
2. Build: write the manifest with the file list, the digest and the SHA-256 of `trusted_hosts.json`, and sign it with a separate release key. The release public key is pinned in `release_signer.pub`.
3. Start-up: verify the manifest signature and that its signer is the pinned key; then re-hash the agent's own files and the trust store and compare. Report modified, missing and unexpected files. Any `.pyc`, `__pycache__`, `.so`, `.pyd` or `.pth` under the package is also a failure: CPython runs a timestamp-based `.pyc` whenever its recorded source mtime and size match, so a planted bytecode file could replace `policy.py` while every source hash still matches. The release is therefore source-only and the agent, demos and tests run with `PYTHONDONTWRITEBYTECODE=1` (`python -B`).
4. Start-up: if `trusted_hosts.json` differs from the sealed copy, always refuse to start (a changed store could add a rogue key). If anything else differs, either refuse to start (`strict_integrity=True`) or start in SAFE MODE, where C13 allows only REQUEST_TELEMETRY_DUMP and PING. The agent also places its *measured* digest in every reply, so peers can see that it is not a trusted release.
5. Every command: compare header.integrity with the sender's trusted digests (C12). A console running modified code is rejected even though its signature and network origin are valid.

3.6 Replay, freshness, lockout and connection limits

Table 3.3 lists the time-based and connection parameters. The replay cache only has to remember an entry until the message could no longer pass C10, so its time to live is the maximum lifetime plus twice the skew. The cache is bounded; if it is ever full and nothing has expired it rejects new messages rather than forgetting live entries.

Table 3.3: Time-based and connection limits

Parameter	Value	Reason
Clock skew window	+/- 60 s	Tolerates normal NTP drift; a captured command goes stale within a minute
Default / maximum lifetime	30 s / 120 s	Commands are interactive; a long-lived signed command is a liability
Replay cache TTL	240 s	max lifetime + 2 x skew; bounded at 100,000 entries, fails closed when full
Lockout threshold / window	5 failures / 60 s	Counted per (address, claimed host); cleared by a request that passes C8
Lockout duration	300 s	Long enough to stop a burst; short enough not to lock out an operator for long
Per-frame deadline	5 s	Whole frame must arrive in time; a byte-by-byte drip cannot hold a worker
Connections per IP	4	More are refused; timeouts and refusals count as L0 failures

Keying the lockout on the address alone would let anyone who can send junk from an address lock out a real host that uses it. Keying it on (address, claimed host), and letting a request that passes the signature check clear the lock, means forged traffic cannot deny service to the real key holder. The cost is that a request to a locked key is still processed up to C8 before it is refused, so a locked-out forger still costs one signature verification per request.

3.7 Fail-closed replies and audit hygiene

A rejected sender receives a reply signed by the agent that says only "request rejected by the CipherMesh trust gate" and an incident number. The checkpoint and reason are kept in the local log and audit trail, so an attacker cannot use the replies to learn which check it failed. The audit trail follows the OWASP logging guidance [11]: one JSON object per decision with time, channel, origin, claimed host, key id, command, checkpoint, reason, latency and the state fingerprint before and after; never the payload, prompt text, patterns, signatures or key material. Strings that come from the attacker are passed through a filter that escapes control characters, so they cannot forge new log lines.

4 IMPLEMENTATION

4.1 Module map

Table 4.1 lists the modules of the new `ciphermesh/trust/` subpackage. Together they are about 3,400 lines of Python. The public API is exported from `ciphermesh/trust/__init__.py` so that the Capstone 5.2 build can import it directly.

Table 4.1: Modules of the ciphermesh/trust subpackage

File	Purpose	Key classes / functions
`canonical.py`	RFC 8785-style canonical JSON; strict parser for untrusted bytes	`canonicalize`, `strict_loads`
`keys.py`	Ed25519 keys, PEM storage, schema-validated trust store, rotation and revocation	`SigningIdentity`, `TrustStore`, `TrustStoreDocument`

File	Purpose	Key classes / functions
`commands.py`	Command vocabulary, roles, STATE_SYNC peer fields, SAFE MODE list	`DEFAULT_ROLES`, `STATE_SYNC_PEER_FIELDS`
`envelope.py`	SignedCommand schema, signing input, sign / parse / parse_and_verify	`create_command`, `parse_and_verify`, `SignedCommand`
`verifier.py`	The ordered checkpoints, replay cache, failure limiter, verdicts	`CommandVerifier`, `Verdict`, `Origin`, `ReplayCache`
`integrity.py`	Release manifest, signing, self-attestation, attestation digest	`build_manifest`, `self_attest`, `measure`
`secure_agent.py`	Agent node: verify, execute on engine, sign replies, audit; spool channel	`SecureCipherMeshAgent`, `SystemCommandChannel`
`secure_server.py`	TCP loop on the Unit 2.2 frame format; C1 on the length prefix; frame deadline and per-IP cap	`serve`, `BackgroundServer`
`client.py`	Signing client with source-address binding; reply verification	`SecureClient`, `verify_reply`
`provision.py`	Generates keys, trust store and signed manifest	`provision`, `ensure_provisioned`
`attack_sim.py`	Breach simulation (Parts A to D) and log writer	`BreachSimulation`, `tamper_copy`
`integrity_check.py`	Host integrity run with eleven tamper experiments	`run`
`benchmark.py`	Sign/verify and per-checkpoint micro-benchmark	`run`, `render`

4.2 Selected code

The core of the cryptographic stage is short. The excerpt below is the real code of checkpoints C8 to C11 (long lines wrapped for print). Each check raises a `_Reject` with a checkpoint id and reason code; `passed()` records the time spent on that checkpoint for the benchmark.

```
# C8 - Ed25519 signature over canonical header + payload.
if not verify_signature(self.trust.public_key(header.key_id),
                        message.signature, message.signing_input()):
    raise _Reject("C8", "BAD_SIGNATURE",
                  "Ed25519 signature does not verify over the canonical bytes")
passed("C8")
# C9 - payload digest bound into the signed header.
if not hmac.compare_digest(payload_digest(message.payload), header.payload_sha256):
    raise _Reject("C9", "PAYLOAD_HASH_MISMATCH", "...")
passed("C9")
# C10 - freshness.
issued, expires = message.issued_at, message.expires_at
if issued > now + self.max_skew:
    raise _Reject("C10", "FUTURE_MESSAGE", ...)
if issued < now - self.max_skew:
    raise _Reject("C10", "STALE_MESSAGE", ...)
if expires <= issued or expires - issued > self.max_ttl:
    raise _Reject("C10", "INVALID_LIFETIME", ...)
if now >= expires:
    raise _Reject("C10", "EXPIRED_MESSAGE", ...)
passed("C10")
# C11 - replay protection (only reached by authentic messages).
problem = self.replay.check_and_add(header.msg_id, header.nonce)
if problem:
    raise _Reject("C11", problem, "message id or nonce already used ...")
```

Listing: checkpoints C8 to C11 (ciphermesh/trust/verifier.py)

Constant-time comparison (`hmac.compare_digest` [12]) is used for the recipient, the host binding of a key, the payload digest and the attestation digest, so that response timing does not reveal how many leading characters of a guess were right. Ed25519 verification itself is done by the `cryptography` library on top of OpenSSL [13].

The agent entry point shows the rule that nothing is executed before the verifier has accepted the frame, and that the state is fingerprinted on both sides of every decision:

```
def handle_frame(self, raw: bytes, origin: Origin, declared_length=None) -> HandleResult:
    before = self.state_summary()
    verdict = self.verifier.verify(raw, origin, declared_length)
    incident = None
    if verdict.accepted:
        status, result = self._execute(verdict)
    else:
        incident = self._next_incident()
        status, result = "REJECTED", {"error": GENERIC_REJECTION}
    after = self.state_summary()
    self._record(verdict, status, result, incident, before, after)
    reply = self._reply(verdict, status, result, incident)   # Ed25519-signed
    ...
```

Listing: SecureCipherMeshAgent.handle_frame (ciphermesh/trust/secure_agent.py)

4.3 Channels and the trusted host environment

TCP loop. `secure_server.py` reads the 4-byte length prefix of the Unit 2.2 format first. If the length is zero or above the limit, it calls the verifier with an empty body so the request is rejected at C1 without reading or allocating the body. The network origin used by C7 is the address returned by `accept()`, which the sender cannot write into the message. The whole frame must arrive within a monotonic 5 s deadline and one address may hold at most four connections; a stalled or excess connection is closed, audited as an L0 failure (FRAME_TIMEOUT or CONNECTION_LIMIT) and counted towards lockout, so slow clients cannot exhaust the worker pool. The client and the simulation bind each socket to a chosen loopback address (127.0.0.10, 127.0.0.66 and so on) before `connect()`, so the agent sees genuinely different peers.

System-command channel. `SystemCommandChannel` processes signed JSON files dropped into a spool directory, or one envelope from standard input. The directory must not be writable by group or others. Each file is opened with `O_NOFOLLOW` and checked with `fstat` on the open descriptor, so the owner and mode belong to exactly the bytes that are read and a symlink is never followed. Processed files are moved to `processed/` or `rejected/` and a signed reply is written to `replies/`.

4.4 Execution on the real engine

An accepted command is turned into a Unit 2.2 `WireMessage` and handed to the existing `CipherMeshAgent.handle()`, so all of the engine's own validation still applies, for example the regular-expression guard on new redaction patterns. The lockdown, pattern-update and telemetry commands become Unit 2.2 command instructions, state synchronisation and inspection keep their Unit 2.2 message types, and the new release-lockdown command clears `force_block_all` under the engine lock. Roles decide who may send what: `security-admin` may lock, release, update patterns, synchronise any policy field, read telemetry and ping; `observer` may only read telemetry and ping; `peer-gateway` may ask for inspections, ping, and synchronise only the four non-weakening fields listed in Algorithm 1.

5 VERIFICATION RUN

5.1 Set-up

The run was produced by `bash demo/run_all.sh`, which provisions fresh keys, runs the integrity check, the breach simulation, the micro-benchmark and the test suite, and writes every file in `logs/`. All timestamps are UTC. Each request travels over a real TCP connection whose client socket is bound to a chosen loopback address: the security console uses 127.0.0.10 (its allowed range is 127.0.0.8/29), the observer 127.0.0.20, the peer gateway 127.0.0.30 and the decommissioned console 127.0.0.40. Attacks come from untrusted addresses (127.0.0.66 to .72, and .99 for the brute-force burst) or from attacker footholds *inside* the admin range (127.0.0.9 and .11 to .15). The attacker key `rogue-console-99` is not in the trust store; the console's rotated-out key is in the store but revoked.

5.2 Legitimate traffic is accepted

Table 5.1 lists the correctly signed requests. All ten were accepted and changed the state only as intended. After S01 a clean prompt was inspected and returned BLOCK, which shows that the lockdown really reached `policy.inspect_text`; S02 released it again.

Table 5.1: Legitimate requests (Parts A, C and D)

ID	Request	Verdict	Effect on engine
S01	Console lockdown over TCP from 127.0.0.10	**ACCEPTED**	lockdown False -> True; clean prompt -> BLOCK
S02	Console releases lockdown	**ACCEPTED**	lockdown True -> False
S03	Observer telemetry dump	**ACCEPTED**	read-only, state unchanged
S04	Console adds pattern `PRJ-[0-9]{4}`	**ACCEPTED**	patterns 0 -> 1
S05	Peer gateway state sync to revision 2	**ACCEPTED**	revision 1 -> 2
S06	Peer gateway inspection (contains an email)	**ACCEPTED**	decision REDACT, state unchanged
S07	Console pattern update via the spool (mode 0600)	**ACCEPTED**	patterns 1 -> 2
S09	Real console from 127.0.0.8 while that key is locked	**ACCEPTED**	passes C8; lockout cleared
S08	Peer gateway inspection after all attacks	**ACCEPTED**	decision REDACT
I02b	Observer telemetry to the SAFE MODE agent	**ACCEPTED**	read-only; reply digest flagged untrusted

5.3 Breach simulation: every unauthorised request is blocked

Table 5.2 lists the 41 attack requests, the checkpoint that stopped each one (column At) and the reason code. Every attack was stopped at the checkpoint it was designed to test, and for every one of them the state fingerprint (lockdown flag, pattern list, policy fields) was identical before and after.

Table 5.2: Attack requests, blocking checkpoint and reason

ID	Attack	At	Reason code	Result
A01	Unsigned command	C2	SCHEMA_VIOLATION	**BLOCKED**

ID	Attack	At	Reason code	Result
A02	Malformed (truncated) JSON frame	C2	JSON_MALFORMED	**BLOCKED**
A03	Unknown header field `role_override` injected	C2	SCHEMA_VIOLATION	**BLOCKED**
A04	Duplicate `command` key (parser differential)	C2	JSON_DUPLICATE_KEY	**BLOCKED**
A05	Length prefix declares 1 MiB	C1	FRAME_TOO_LARGE	**BLOCKED**
A06	Protocol downgrade to cm-sig/0	C3	UNSUPPORTED_VERSION	**BLOCKED**
A07	Valid command addressed to node-02	C4	WRONG_RECIPIENT	**BLOCKED**
A08	Unknown host with its own key	C5	UNKNOWN_HOST	**BLOCKED**
A09	Decommissioned host, still-valid key	C5	HOST_REVOKED	**BLOCKED**
A10	Spoofed host_id with attacker key	C6	KEY_NOT_BOUND	**BLOCKED**
A11	Rotated-out (revoked) admin key	C6	KEY_REVOKED	**BLOCKED**
A12	Stolen valid admin key from 127.0.0.66	C7	ORIGIN_NOT_ALLOWED	**BLOCKED**
A13	Forged signature, admin key_id, inside admin range	C8	BAD_SIGNATURE	**BLOCKED**
A14	Payload tampered after signing	C8	BAD_SIGNATURE	**BLOCKED**
A15	Payload tampered, digest recomputed	C8	BAD_SIGNATURE	**BLOCKED**
A16	Trusted key, header digest does not match body	C9	PAYLOAD_HASH_MISMATCH	**BLOCKED**
A17	Stale command issued 10 minutes ago	C10	STALE_MESSAGE	**BLOCKED**
A18	Expired command (ttl 5 s, sent after 20 s)	C10	EXPIRED_MESSAGE	**BLOCKED**
A19	Future-dated command (+5 minutes)	C10	FUTURE_MESSAGE	**BLOCKED**
A20	Byte-identical replay of accepted S01	C11	REPLAY_MSG_ID	**BLOCKED**
A21	Console running backdoored policy.py	C12	ATTESTATION_MISMATCH	**BLOCKED**
A22	Observer key asks for lockdown	C13	COMMAND_NOT_PERMITTED	**BLOCKED**
A23	World-writable command file in spool	C7	LOCAL_SOURCE_UNTRUSTED	**BLOCKED**
A24	Forged command file in spool	C8	BAD_SIGNATURE	**BLOCKED**
A25.1–5	Brute-force burst from 127.0.0.99, attempts 1 to 5	C7	ORIGIN_NOT_ALLOWED	**BLOCKED**
A25.6–8	Same burst, attempts 6 to 8 after lockout	L0	SOURCE_LOCKED_OUT	**BLOCKED**

ID	Attack	At	Reason code	Result
A26	Peer STATE_SYNC switches off secret blocking	C13	STATE_FIELD_NOT_PERMITTED	**BLOCKED**
A27.1–5	Forged burst to lock out the console (127.0.0.8)	C8	BAD_SIGNATURE	**BLOCKED**
A28.1	4 slow-drip connections from 127.0.0.72	L0	FRAME_TIMEOUT	**BLOCKED**
A28.2	5th connection from the same address	L0	CONNECTION_LIMIT	**BLOCKED**
I02a	Valid admin lockdown to a SAFE MODE agent	C13	SAFE_MODE_RESTRICTED	**BLOCKED**

Figure 5.1 summarises the same result by checkpoint. Every one of the thirteen checkpoints and the lockout gate stopped at least one real attack, so none of them is dead code. C8 stopped the most requests (nine), because the lockout-abuse burst A27 and several forgeries came from inside the admin range; C7 stopped seven, mostly the brute-force burst from outside every allowed range. The five C1 and C2 rejections never reach any cryptography at all. After A27 the attacker had locked the key (127.0.0.8, sec-console-01), yet the real console from the same address was accepted at once (S09) because its request passed C8 and cleared the lock.

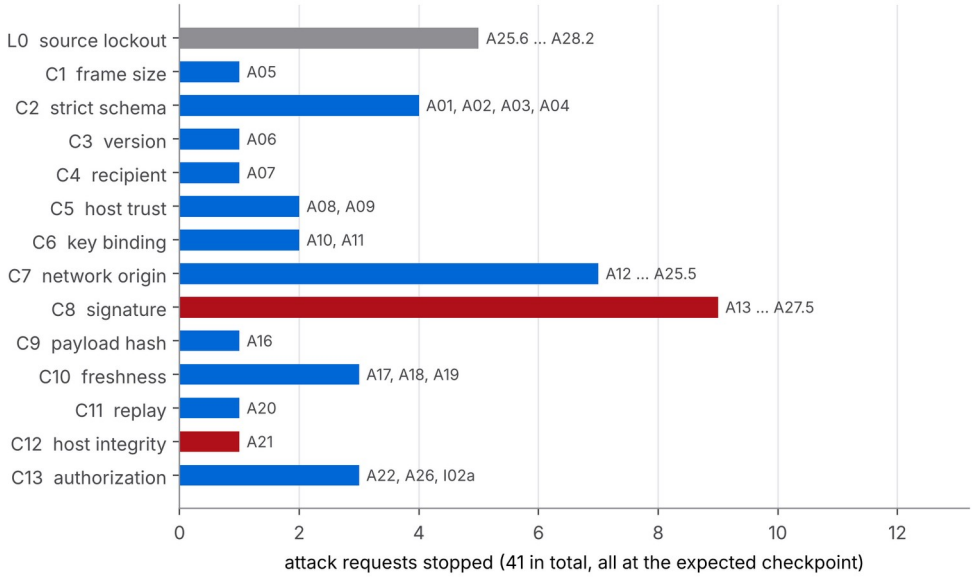


Figure 5.1: Attack requests stopped by each checkpoint (labels give attack ids)

Figure 5.2 contrasts the accepted path (S01) with the forged-signature attack (A13). Both messages carry the same command and the same admin key id; the forged one passes C1 to C7 because it comes from inside the admin range, and is stopped by the signature check. The engine is never called.

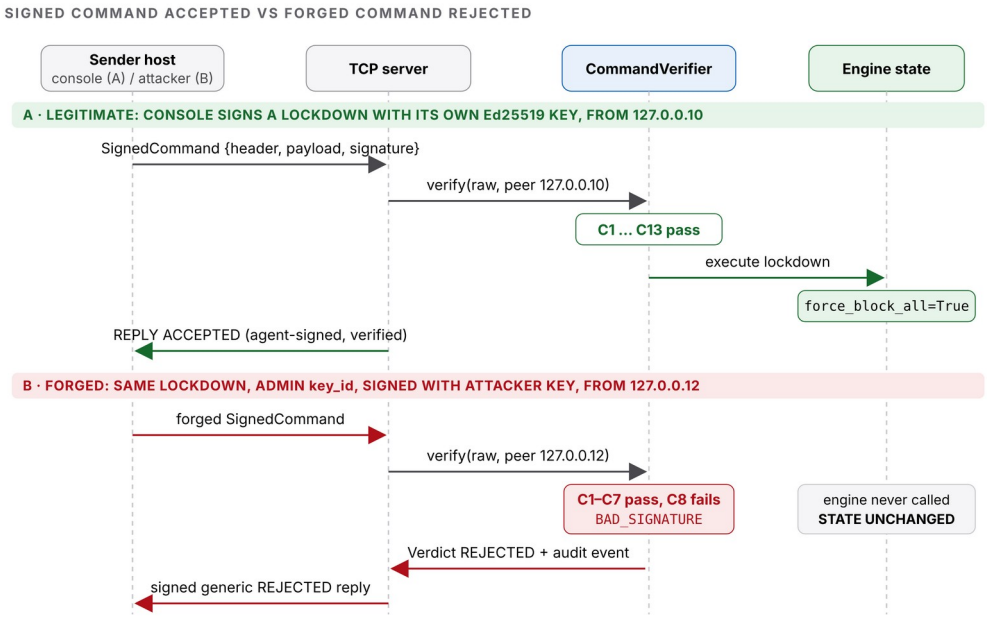


Figure 5.2: Signed command accepted versus forged command rejected at C8

5.4 A blocked unauthorised request in the runtime log

The excerpt below is copied from `logs/security_runtime.log` (the date prefix is shortened to the time of day; long lines are wrapped). It shows the request, the agent's own BLOCKED UNAUTHORIZED REQUEST line, the verdict with checkpoint and reason, the state fingerprint before and after, and the agent-signed generic reply.

```
16:27:08.909Z INFO  [A20] Replay of the captured S01 lockdown  (ATTACK)
16:27:08.909Z INFO    request : tcp 127.0.0.11 -> agent | claimed_host=sec-console-01
                      key_id=98417bb2a3867e6c command=TRIGGER_EMERGENCY_LOCKDOWN
16:27:08.909Z INFO    expected: REJECT at C11 REPLAY (REPLAY_MSG_ID)
16:27:08.911Z WARN  [AGENT] BLOCKED UNAUTHORIZED REQUEST INC-00020:
                      command=TRIGGER_EMERGENCY_LOCKDOWN claimed_host=sec-console-01
                      origin=tcp://127.0.0.11:40085 checkpoint=C11 REPLAY
                      reason=REPLAY_MSG_ID
16:27:08.913Z INFO    verdict : REJECTED at C11 REPLAY reason=REPLAY_MSG_ID
                      latency=1151 us incident=INC-00020
16:27:08.913Z WARN    >>> BLOCKED UNAUTHORIZED REQUEST: TRIGGER_EMERGENCY_LOCKDOWN
                      via 127.0.0.11 stopped at C11 (REPLAY_MSG_ID)
16:27:08.913Z INFO    state   : fingerprint 51c3f3e8e21258c5 -> 51c3f3e8e21258c5
                      lockdown=False patterns=2 -> STATE UNCHANGED
16:27:08.913Z INFO    reply   : agent-signed REJECTED reply, signature VERIFIED;
                      body is a generic error (no checkpoint leaked to sender)
16:27:08.913Z INFO    result  : PASS
```

Listing: replay attack A20 as recorded in logs/security_runtime.log

The same event is stored in the audit trail as one JSON object. The first rejected event of the run (A01) is shown below; note that it contains metadata only.

```
{"ts": "2026-11-02T16:27:08.846Z", "event": "COMMAND_REJECTED",
 "checkpoint": "C2", "checkpoint_name": "STRICT_SCHEMA",
 "reason": "SCHEMA_VIOLATION", "detail": "envelope is missing ['signature']",
 "incident": "INC-00001", "status": "REJECTED", "agent_id": "ciphermesh-node-01",
 "channel": "tcp", "origin": "tcp://127.0.0.66:36775", "latency_us": 87.4,
 "state_before": "51c3f3e8e21258c5", "state_after": "51c3f3e8e21258c5",
 "state_changed": false}
```

Listing: an audit event from logs/security_events.jsonl

Four security alerts were raised during the run: HOST_INTEGRITY_FAILURE for A21 (console digest 55ac241b22133037... is not a trusted release), and SOURCE_LOCKOUT for the keys (127.0.0.99, sec-console-01) after A25.5, (127.0.0.8, sec-console-01) after A27.5, and (127.0.0.72, -) after the slow connections of A28.

5.5 State is unchanged and prompts are still processed correctly

The baseline fingerprint taken after Part A was `51c3f3e8e21258c5` (lockdown off, two custom patterns, policy revision 2), and the fingerprint after all 41 attacks was the same. Part C then inspected six prompts directly on the engine (Table 5.3); all decisions were correct, including both custom patterns added by the legitimate admin commands. A signed INSPECT over TCP (S08) also returned the expected REDACT.

Table 5.3: Prompt decisions after the breach simulation

Prompt	Content	Expected	Got
Summarise the Q3 hiring plan	clean text	ALLOW	**ALLOW**
Send the invoice to rahul.verma@...	email (D5)	REDACT	**REDACT**
Status of PRJ-2041 ...	pattern added in S04	REDACT	**REDACT**
Payroll query for EMP-204118	pattern added in S07	REDACT	**REDACT**
Use AKIAIOSFODNN7EXAMPLE to call S3	AWS key (D1)	BLOCK	**BLOCK**
See https://wiki.corp/finance/plan	internal URL (R3)	REVIEW	**REVIEW**

5.6 Host integrity results

`logs/integrity_check.log` records eleven integrity experiments, each on a temporary copy so the real tree is never changed (Table 5.4). The clean tree of 44 source files matched the signed manifest in 5.3 ms, and all eleven tampering experiments were detected. IC11 was added after a review showed that a planted `__pycache__/policy.cpython-313.pyc`, carrying the clean source's mtime and size, is loaded by Python although every `.py` file still matches; the log shows the backdoor really running (an AWS key classified ALLOW) and the attestation now failing on the bytecode files.

Table 5.4: Host integrity checks (logs/integrity_check.log)

ID	Experiment	Outcome	Result
IC1	Verify release manifest signature	signature valid	**PASS**
IC2	Self-attest the running package	NORMAL mode, 44 files match	**PASS**
IC3	Backdoor injected into policy.py	MODIFIED policy.py -> SAFE MODE	**PASS**
IC4	Unexpected file trust/evil_hook.py planted	UNEXPECTED file -> SAFE MODE	**PASS**
IC5	trust/verifier.py deleted	MISSING file -> SAFE MODE	**PASS**
IC6	Manifest edited to approve the backdoor	signature INVALID	**PASS**
IC7	Manifest re-signed with an attacker key	not the pinned release key	**PASS**
IC8	Rogue admin host added to trust store copy	hash mismatch -> start refused (any mode)	**PASS**
IC9	Strict start-up on the backdoored tree	IntegrityError, start refused	**PASS**

ID	Experiment	Outcome	Result
IC10	Digest a compromised console would send	not trusted -> C12 rejects	**PASS**
IC11	Backdoored .pyc planted, all .py unchanged	BYTECODE artefacts -> SAFE MODE	**PASS**

Part D of the breach simulation repeated the self-attestation test on a live agent: in strict mode it refused to start; in normal mode it started in SAFE MODE, refused a valid admin lockdown at C13 (SAFE_MODE_RESTRICTED, I02a), still served observer telemetry (I02b), and its replies carried a digest that the client flagged as untrusted. The corresponding runtime log lines are "self-attestation FAILED", "agent REFUSED TO START" and "entering SAFE MODE".

5.7 Timeline and cost

Figure 5.3 plots every request of the run in order with its verifier latency. Accepted requests take between 0.45 and 1.0 ms in this run (the first requests include interpreter warm-up); requests rejected at C1 or C2 cost under 0.1 ms. The lockout rejections A25.6 to A25.8 cost 0.13 to 0.36 ms, because a request to a locked key is still checked up to C7 or C8 so that an authentic request can clear the lock. The two A28 cases are connection-level events with no verifier run and are not plotted.

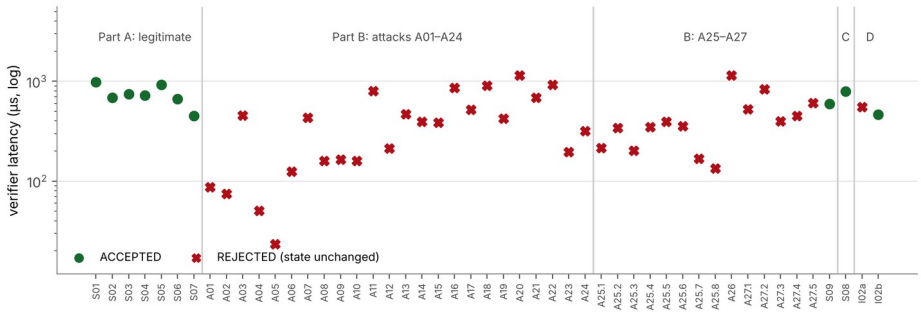


Figure 5.3: Breach simulation timeline: verifier latency per request

Table 5.5 gives the main numbers from `logs/crypto_benchmark.txt` (3,000 iterations; medians are reported because the shared test machine produced occasional long-tail outliers). Figure 5.4 breaks the accepted path into checkpoints: C8 (canonical re-encoding plus Ed25519 verification) is about 68 % of the mean cost and strict parsing at C2 about 17 %; the remaining twelve steps together cost about 15 %.

Table 5.5: Micro-benchmark results (median of 3,000 runs unless stated)

Operation	Median	Note
Ed25519 sign / verify (564-byte input)	50.2 / 162.2 us	sign on client, verify at C8
HMAC-SHA256 sign / verify (Unit 2.2)	2.2 / 2.2 us	no attribution possible
CommandVerifier.verify, accepted (13/13)	301.4 us	3,021 commands/s (mean)
Reject at C1 / C2 / C7 / C8	5 / 12 / 61 / 345 us	early checks are cheap
Self-attestation (44 files + signature)	4.9 ms	once at start-up
TCP round trip, signed command + reply	2.92 ms	new connection each time

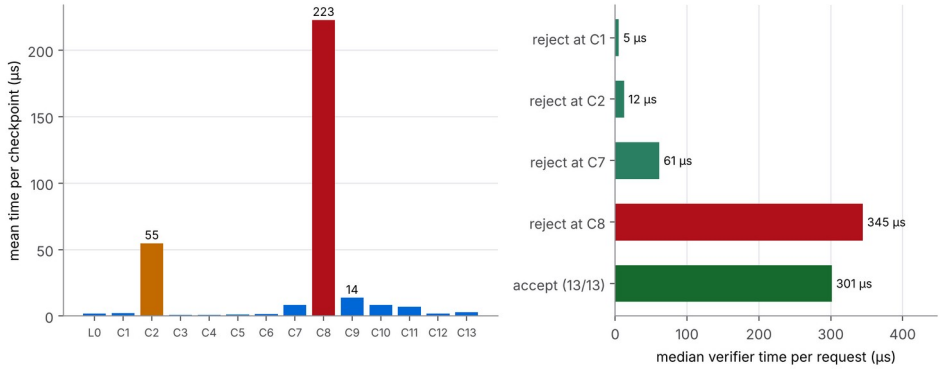


Figure 5.4: Per-checkpoint cost (left) and early rejection versus acceptance (right)

Ed25519 verification is roughly 70 times slower than an HMAC check, but at well under a millisecond per command it is negligible next to an LLM call, and commands to a gateway are rare compared with prompts. The extra cost buys attribution, least privilege and the guarantee that a compromised gateway cannot forge commands.

6 TESTING

6.1 Test suite

The full suite contains 269 tests: the 112 engine tests from earlier units, which still pass unchanged, and 157 new tests in `tests/trust/`. Table 6.1 lists the new files. The output of the run is saved in `logs/pytest_results.txt` (269 passed in 12.80 s). The session runs with bytecode writing disabled, because the release is source-only.

Table 6.1: New tests in tests/trust

File	Tests	What is covered
`test_canonical.py`	35	key order, UTF-16 sorting, escapes, ECMAScript number vectors, invalid values, stability, strict parser incl. 5000-digit and 1e400 numbers
`test_keys.py`	22	key id rule, encrypted PEM (0600), wrong passphrase, 13 schema violations, unsafe files, rotation and revocation
`test_envelope.py`	20	round trip, wire-format independence, tampering of 7 header fields, domain separation, payload schemas
`test_verifier.py`	45	every role accepted; a negative test for L0 and each of C1-C13; STATE_SYNC field rule; clock skew; replay; cache TTL; ordering; per-host lockout and clearing; internal fault
`test_integrity.py`	12	manifest coverage, signature tamper, foreign signer, backdoor, missing and extra files, trust store change, planted .pyc, .so and .pth
`test_secure_agent.py`	13	lockdown on the real policy, engine regex guard, peer cannot weaken policy, generic replies, audit hygiene, SAFE MODE, trust store refusal, spool, symlink, FIFO, vanished file
`test_e2e_tcp.py`	7	real TCP with source binding: origin, oversized and malformed frames, replay, 16 concurrent clients, half-open socket, frame deadline and per-IP cap
`test_demos.py`	3	full breach simulation, integrity run (11/11) and benchmark end to end

6.2 Edge cases worth noting

- **Clock skew** is tested from both sides: a sender 55 s fast or slow is accepted, 61 s is rejected, and a receiver whose clock runs 90 s ahead rejects a fresh message as stale.
- **Replay cache poisoning**: a forged message that reuses the msg_id and nonce of a valid message is rejected at C8 and does not prevent the valid message from being accepted afterwards.
- **First failure wins**: a message that is wrongly addressed, from an unknown host, forged and stale is reported at C4, proving the order is enforced.
- **Lockout** is keyed on (address, claimed host): five forged admin commands lock that key, further forgeries and badly addressed commands are refused at L0, but a correctly signed command from the same address passes C8 and clears the lock. Junk sent from an address does not lock out a real host using it.
- **Policy weakening**: a peer gateway STATE_SYNC that touches custom patterns, secret blocking, PII redaction, internal-URL review or lockdown is refused at C13, and an AWS key is still classified BLOCK afterwards.
- **Audit hygiene** is asserted directly: after an INSPECT with an email address and a pattern update, the audit file contains neither the address, nor the pattern, nor any signature or key material. **Fail closed**: an exception inside a checkpoint becomes INTERNAL_ERROR without leaking its text.

7 SUBMISSION ANALYSIS

7.1 Requirement, evidence and status

Table 7.1: Requirement status

Requirement	Evidence	Status
Signature-based secure payload parsing	envelope.py + verifier C2/C5/C6/C8/C9; 63 envelope/verifier tests	**PASS**
Sender public keys verified before processing	C5/C6 bind key to host; agent holds public keys only	**PASS**
Trusted host environment for network loop	C7 on kernel peer IP; A12, A25 rejected	**PASS**
Trusted host environment for system commands	C7 owner/mode/O_NOFOLLOW; A23, A24 rejected	**PASS**
Host integrity verification	self-attestation incl. bytecode, SAFE MODE, C12; 11/11 checks	**PASS**
Untrusted input breach simulated and handled safely	41/41 attacks blocked at expected checkpoint, state unchanged	**PASS**
Runtime logs with blocked unauthorised request	41 BLOCKED summary lines in security_runtime.log; 64 audit events	**PASS**
Existing engine untouched and passing	112 engine tests pass	**PASS**

7.2 Strengths

- A gateway can no longer forge commands: it only holds public keys. One host can be revoked or rotated without touching the others.
- Every checkpoint is exercised by at least one real attack over real sockets, and every rejection is shown to leave the engine state byte-for-byte unchanged.

- Defence in depth is visible in the results: a stolen key from outside is stopped by C7, a foothold inside the admin range by C8, a captured message by C10/C11 and a compromised console by C12.

7.3 Honest limitations

- **Software attestation only.** A host that is fully compromised could report the trusted digest while running other code. Closing this needs a hardware root of trust, such as TPM 2.0 quotes and measured boot [14], which is outside the scope of a student lab.
- **Loopback, not a real network.** Source addresses are real kernel-reported addresses on 127.0.0.0/8, but IP-based origin checks can be spoofed on some real networks; in production C7 should be backed by mutual TLS or a VPN, with Ed25519 still providing message-level authenticity.
- **Replay state is in memory.** A restart clears the cache. Because C10 rejects anything older than 60 s, a replay after restart is only possible inside that small window; a persistent cache would close it.
- **Demo passphrase.** When `CIPHERMESH_KEY_PASSPHRASE` is not set, provisioning uses a built-in demo passphrase and logs a warning. Private keys are generated locally and are not part of the submission.
- **Lockout residual risk.** An attacker inside an allowed range can still lock a (address, host) key for up to 300 s with forged traffic; the real host clears it with its next authentic request, but forged traffic to a locked key still costs one signature check each. A source-only release also means start-up must run with bytecode writing disabled; a deployment that wants bytecode should use checked-hash `.pyc` files sealed in the manifest.
- **Trust anchor on disk.** `release_signer.pub` and the trust store are protected by file permissions and the sealed hash; production would keep them in an HSM or configuration service.

7.4 How this feeds the Capstone 5.2

The subpackage has a stable public API (`SecureCipherMeshAgent`, `CommandVerifier`, `create_command`, `TrustStore`, `self_attest`). The capstone can put its whole control plane behind the same verifier: provider-routing updates, review approvals and policy pushes become signed commands with their own roles, and the audit trail format can be reused directly in the capstone dashboard.

8 CONCLUSION

This unit replaced a shared-secret trust model with per-host Ed25519 identities, a strict and canonical envelope, an ordered set of thirteen fail-closed checkpoints and software host-integrity attestation. In a breach simulation over real TCP and a local command channel, all ten legitimate requests were accepted and all 41 unauthorised ones were blocked at the expected checkpoint, with the agent state unchanged and prompts still classified correctly. The integrity run detected all eleven tampering experiments, including a planted bytecode cache. The 112 earlier tests and 157 new tests pass. CipherMesh now protects not only the prompts that pass through it, but also the commands that control it.

REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," Internet Engineering Task Force, RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[3] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," Internet Engineering Task Force, RFC 8032, Jan. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8032

[4] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77-89, 2012.

[5] E. Barker, "Recommendation for Key Management: Part 1 - General," NIST Special Publication 800-57 Part 1 Rev. 5, National Institute of Standards and Technology, May 2020, doi: 10.6028/NIST.SP.800-57pt1r5.

[6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[7] G. Klyne and C. Newman, "Date and Time on the Internet: Timestamps," Internet Engineering Task Force, RFC 3339, Jul. 2002. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3339

[8] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," Internet Engineering Task Force, RFC 8785, Jun. 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8785

[9] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) Architecture," Internet Engineering Task Force, RFC 9334, Jan. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9334

[10] OWASP Foundation, "A08:2021 - Software and Data Integrity Failures," *OWASP Top 10:2021*. Accessed: Nov. 2, 2026. [Online]. Available: https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/

[11] OWASP Foundation, "Logging Cheat Sheet," *OWASP Cheat Sheet Series*. Accessed: Nov. 2, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

[12] Python Software Foundation, "hmac - Keyed-Hashing for Message Authentication," *Python 3.13 Documentation*. Accessed: Nov. 2, 2026. [Online]. Available: https://docs.python.org/3/library/hmac.html

[13] Python Cryptographic Authority, "Ed25519 signing," *cryptography documentation*. Accessed: Nov. 2, 2026. [Online]. Available: https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ed25519/

[14] Trusted Computing Group, *Trusted Platform Module Library Specification, Family "2.0"*, Level 00, Revision 01.59, Nov. 2019.

APPENDIX A HOW TO RUN AND FILE LISTING

A.1 Commands

All commands are run from the deliverable root. The project Python environment needs `cryptography`, `pydantic` and `pytest` (see `requirements.txt`).

```
# everything at once (provision, integrity, breach simulation, benchmark, tests)
bash demo/run_all.sh python3

# or step by step (source-only release: never write bytecode)
export PYTHONDONTWRITEBYTECODE=1
python -B -m ciphermesh.trust.provision       # keys, trusted_hosts.json, manifest
python -B demo/run_integrity_check.py         # -> logs/integrity_check.log
python -B demo/run_breach_simulation.py       # -> logs/security_runtime.log,
                                              #    security_events.jsonl,
                                              #    breach_simulation_summary.*
python -B demo/run_crypto_benchmark.py        # -> logs/crypto_benchmark.txt/.json
python -B -m pytest tests -o addopts="" -v --no-header > logs/pytest_results.txt

# interactive use (keys are provisioned on first run)
python -B -m ciphermesh.trust.secure_server --port 9061
python -B -m ciphermesh.trust.client --as sec-console-01 --source 127.0.0.10 \
       TRIGGER_EMERGENCY_LOCKDOWN --payload '{"reason": "drill"}'
python -B -m ciphermesh.trust.secure_agent --spool /path/to/spool   # local channel
```

Listing: commands that reproduce every log

A.2 Deliverable files

Table A.1 lists the files of the submission.

Table A.1: Deliverable files

Path	Content
`ciphermesh/trust/*.py`	the 13 modules of Table 4.1 plus `__init__.py`; no bytecode
`ciphermesh/trust/trusted_hosts.json`	trust store from the final provisioning run (public data only)
`ciphermesh/trust/release_manifest.json`, `release_signer.pub`	signed release manifest and pinned release key
`ciphermesh/` (other modules)	Units 1-3 engine, unchanged
`tests/engine/`, `tests/trust/`	112 engine tests and 157 new tests
`demo/run_all.sh`, `demo/run_*.py`	scripts that produce the logs
`logs/security_runtime.log`	readable runtime log of the breach simulation
`logs/security_events.jsonl`	agent security audit trail (64 events)
`logs/breach_simulation_summary.json` / `.txt`	per-case results and summary table
`logs/integrity_check.log`	host integrity run (11 checks)
`logs/crypto_benchmark.txt` / `.json`	micro-benchmark results
`logs/pytest_results.txt`	full test run, 269 passed
`docs/figures/`	figures, `diagrams.html` source, `make_charts.py`
`docs/report_src/`	`content.js` source of this report and build notes